

1. Estrategia: Algoritmo de Resolución

Sigue este orden lógico en el examen para no perderte.

Acción	Descripción
<code>const elemento = ...</code>	Guarda en variables todos los elementos del HTML (inputs, botones, zonas de resultado) al principio del script.
<code>.innerHTML = ""</code>	Si vas a pintar una lista nueva, vacía siempre el contenedor antes de empezar a añadir elementos para no duplicar .
<code>.addEventListener(...)</code>	Define qué acción del usuario (clic, cambio, envío) dispara tu lógica .
Datos (Array/Map)	Realiza los cálculos, validaciones o filtrado de datos antes de pintar nada.
<code>createElement / append</code>	Crea los elementos HTML, dales contenido y añádelos a la página.

2. Acceso al DOM (Selectores y Lectura)

Cómo "agarrar" elementos y leer su contenido.

Método / Propiedad	Uso Recomendado	Retorno / Nota
<code>document.getElementById('id')</code>	Para elementos únicos (botones principales, contenedores).	Un solo elemento.

<code>document.querySelector('.css')</code>	Para clases, selectores complejos o el primer elemento de un tipo.	Un solo elemento (el primero que encuentra).
<code>document.querySelectorAll('.css')</code>	Para capturar grupos (radios, listas de items).	NodeList (Lista). Debes usar <code>forEach</code> para recorrerla.
<code>document.forms</code>	Colección de todos los formularios.	Permite acceder por índice o nombre (<code>forms['miForm']</code>).
<code>.value</code>	Lectura: Inputs de texto, números, selects.	Siempre devuelve un String.
<code>.textContent</code>	Lectura: Etiquetas de texto (p, li, span).	Devuelve solo el texto sin etiquetas.
<code>.innerHTML</code>	Lectura/Escritura: Contenido interno.	Interpreta y renderiza etiquetas HTML.
<code>.checked</code>	Lectura: Checkboxes y Radio button.	Devuelve true si está marcado, false si no.

3. Estructuras de Datos (La Memoria)

Dónde guardar la información mientras funciona el programa.

Estructura	Definición	Métodos Clave
------------	------------	---------------

Array []	Lista ordenada donde importa la posición (0, 1, 2...)	• .push(x): Añade al final . • .filter(f): Crea un array nuevo filtrado . • .includes(x): true si existe (evita duplicados) . • .map(f): Transforma elementos . • .sort(): Ordena la lista
Set new Set()	Colección de valores únicos. No permite duplicados.	• .add(v): Añade valor . • .has(v): Comprueba si existe . • .delete(v): Borra un valor.

Map new Map()	Diccionario Clave-Valor. Asocia datos (ej: Nombre -> Teléfono).	• .set(k, v): Guarda valor asociado a clave . • .get(k): Recupera el valor . • .has(k): Verifica si la clave existe.
---------------	---	--

4. Eventos (La Interacción)

Cómo reacciona la página al usuario. Recuerda la estructura:
elemento.addEventListener('evento', funcion).

Evento	Se dispara cuando...	Acción Crítica / Truco
click	Se pulsa y suelta el ratón sobre un elemento.	Es el evento estándar para botones y enlaces.
submit	Se intenta enviar un formulario.	¡IMPORTANTE! Usa e.preventDefault() para evitar que la página se recargue y pierdas los datos.
change	El valor cambia y el elemento pierde el foco (se confirma el cambio).	Ideal para <select>, <input type="checkbox"> o <input type="radio">.
keydown /	Se pulsa (baja) o se suelta (sube) una tecla.	Usa e.key para saber qué tecla fue (ej: "Enter" para enviar sin ratón).

keyu p		
focus	El usuario entra en un campo (el cursor parpadea dentro).	Útil para limpiar mensajes de error o mostrar ayudas visuales al usuario.
blur	El usuario sale de un campo (clic fuera o tabulador).	Útil para validar el dato justo cuando terminan de escribir (ej. comprobar email).

El Objeto Evento (e)

Es el parámetro que recibe tu función (puedes llamarlo **e** o **event**). Contiene toda la información de lo que acaba de pasar.

- **e.target**: El elemento exacto del HTML que originó el evento (el botón que pulsaste, el input que escribiste).
- **e.preventDefault()**: Cancela el comportamiento por defecto del navegador (evita que un formulario recargue la web o que un enlace te lleve a otra página).
- **e.stopPropagation()**: Evita que el evento "suba" (burbujeo) a los elementos padre, para que no se activen eventos de contenedores superiores.
-

5. Manipulación Dinámica (Renderizado)

Cómo crear y modificar la página desde JavaScript.

Código JS	Descripción
<code>document.createElement('tag')</code>	Crea un nodo HTML en memoria (ej: li, div).
<code>elem.textContent = "Texto"</code>	Rellena el texto del elemento.

<code>elem.classList.add('clase')</code>	Añade una clase CSS (mejor que modificar <code>.style</code> directo).
<code>elem.classList.toggle('clase')</code>	Pone o quita la clase automáticamente.
<code>elem.setAttribute('src', 'x.jpg')</code>	Modifica atributos HTML (como imágenes o enlaces).
<code>padre.appendChild(hijo)</code>	Pone el elemento creado dentro del padre (al final).
<code>padre.removeChild(hijo)</code>	Elimina un elemento específico del DOM.

6. Propiedades Específicas de Formularios

Controles útiles para exámenes con validaciones.

Propiedad	Descripción
<code>input.disabled = true</code>	Desactiva el control para que no se pueda usar.
<code>form.reset()</code>	Borra todos los campos del formulario volviendo al estado inicial.
<code>input.focus()</code>	Pone el cursor automáticamente dentro del input.
<code>input.type</code>	Devuelve el tipo de input (text, password, checkbox).

7. Matemáticas y Fechas (Herramientas Auxiliares)

Útiles si te piden generar datos aleatorios o fechas actuales.

Categoría	Método	Uso
Aleatorio	<code>Math.random()</code>	Genera número entre 0 y 1.
Redondeo	<code>Math.floor(x) / Math.ceil(x)</code>	Redondea hacia abajo / hacia arriba.
Entero Random	<code>Math.floor(Math.random() * 10)</code>	Truco para sacar un entero del 0 al 9.
Fecha	<code>new Date()</code>	Crea objeto con fecha/hora actual.
Datos Fecha	<code>.getFullYear() / .getMonth()</code>	Obtiene el año o el mes (ojo: mes es 0-11).

○

¡Claro! Aquí tienes el apartado de "Organización Anti-Bloqueo" diseñado para que encaje perfectamente con el estilo de tu guía. Son estrategias prácticas para cuando te quedas en blanco o el código no funciona.

8. Organización Anti-Bloqueo

Técnicas para desatascarse cuando el código no funciona o no sabes por dónde seguir.

1. Divide y Vencerás:

No intentes resolver todo el ejercicio en una sola función gigante. Rompe el problema en trozos pequeños (ej. primero capturar datos, luego validar, luego pintar). Si un trozo funciona, pasa al siguiente.

2. El poder del `console.log`:

No programes a ciegas. Pon un `console.log("llegué aquí")` o `console.log(variable)` después de cada paso importante para confirmar que los datos son lo que esperas.

- ¿No sale nada en consola? → Revisa si el evento está bien vinculado.
- ¿Sale `null`? → Revisa el `getElementById` o el selector.

3. Lee la Consola (F12):

El navegador es tu chivato. Si sale texto rojo, léelo. Generalmente te dice el tipo de error (ej. Uncaught TypeError) y el número de línea. Ve directo a esa línea.

4. Revisa la sintaxis básica (Checklist de pánico):

- ¿Has cerrado todas las llaves {} y paréntesis ()?
- ¿Has escrito document o console en minúsculas?
- ¿Coinciden los IDs del HTML con los de tu querySelector?

5. Pseudocódigo en comentarios:

Si te bloqueas con la lógica, deja de escribir código. Escribe en comentarios (//) paso a paso lo que quieres hacer en lenguaje natural.

Ejemplo: // 1. Recorro la lista. // 2. Si el precio es mayor a 50, lo sumo.

Luego, traduce cada línea a JavaScript.

6. Pausa de 1 minuto:

Si llevas 15 minutos con el mismo error, levanta las manos del teclado, respira y bebe agua (o dale un muerdo a tu bocadillo de salchichón para recargar pilas). A veces el error es un simple error tipográfico que solo ves al volver a mirar con ojos frescos.